

□ BUSIT — Complete System Documentation

What is this document? This is the full technical documentation for the BUSIT (Bus Credit System) web application. It explains *every single part* of the project — from the very first HTML pixel the user sees, to the database query that stores their data, to the email that lands in their inbox. It is written so that someone with zero prior knowledge can understand it, while also being detailed enough to satisfy a technical interviewer.

Table of Contents

1. [Project Overview — The Big Picture](#)
 2. [System Architecture — How Everything Connects](#)
 3. [Technology Stack — What Tools We Used and Why](#)
 4. [Project File Structure — Where Everything Lives](#)
 5. [The Frontend — What the User Sees](#)
 6. [The Backend — What Happens Behind the Scenes](#)
 7. [The Database — Where Data is Stored](#)
 8. [Authentication & Security — Keeping Users Safe](#)
 9. [The QR Code System — The Core Innovation](#)
 10. [The Payment Flow — How Money Moves](#)
 11. [The Email Notification System](#)
 12. [Deployment & Infrastructure](#)
 13. [Complete Request Lifecycle — A Full Journey](#)
 14. [Interview Questions & Answers](#)
-

1. Project Overview — The Big Picture

What problem does BUSIT solve?

Imagine you are a university student. Every day, you take a campus shuttle bus. Currently, you either pay cash (which the driver has to count and manage) or use a physical card (which requires expensive hardware readers bolted into every bus).

BUSIT eliminates both problems. It replaces physical cash and hardware card readers with a purely software-based digital wallet system that runs entirely in a web browser.

How does it work in plain English?

1. A **Student** creates an account on the BUSIT website.

2. The student adds credits (digital money) to their wallet.
3. When they board the shuttle, they open the BUSIT website on their phone, which displays a **QR code** (a square barcode) unique to them.
4. The **Driver** also has BUSIT open on their phone or tablet. Their screen shows a live camera. They point the camera at the student's QR code.
5. The system instantly reads the QR code, identifies the student, deducts the fare (20 credits) from their wallet, and sends them an email receipt.

No cash. No hardware. Just two phones and a web browser.

Who are the users?

User Type	What They Do
Student	Registers, adds credits, shows QR code to board the shuttle, receives email receipts
Driver	Registers, uses the live camera scanner on their dashboard to scan student QR codes and process fare payments

2. System Architecture — How Everything Connects

High-Level Architecture Diagram

```
graph TB
    subgraph "User's Device (Browser)"
        A["Student's Phone<br/>Shows QR Code"]
        B["Driver's Phone<br/>Camera Scanner"]
    end

    subgraph "Vercel Cloud (Serverless)"
        C["Next.js Frontend<br/>(React Pages)"]
        D["Next.js API Routes<br/>(Backend Logic)"]
        E["NextAuth.js<br/>(Authentication)"]
    end

    subgraph "External Services"
        F["Neon PostgreSQL<br/>(Cloud Database)"]
        G["Gmail SMTP<br/>(Email Service)"]
    end

    A -->|"HTTPS Request"| C
    B -->|"HTTPS Request"| C
    C -->|"fetch() calls"| D
    D -->|"Session Check"| E
    D -->|"Prisma ORM Queries"| F
    D -->|"Nodemailer"| G
```

What does this diagram mean?

Think of the architecture as a restaurant:

- **The Browser** is the customer sitting at a table. They can see the menu (the UI) and place orders (click buttons).
- **Next.js Frontend** is the waiter. It takes the customer's order and walks it to the kitchen.
- **Next.js API Routes** are the kitchen. This is where the actual cooking (business logic) happens. The kitchen decides: "Does this customer have enough money? Is their card blocked? Should I send them a receipt?"
- **NextAuth.js** is the bouncer at the door. Before anyone enters the restaurant, the bouncer checks their ID (session token) to verify they are who they claim to be.
- **Neon PostgreSQL** is the filing cabinet in the back office. Every customer's account, balance, and transaction history is stored here permanently.
- **Gmail SMTP** is the mail room. After the kitchen processes an order, it asks the mail room to send a receipt to the customer's email address.

Why is this architecture good?

Benefit	Explanation
Separation of Concerns	The frontend (what users see) is completely separate from the backend (business logic). If you redesign the UI, the backend doesn't change.
Serverless Scalability	Vercel doesn't run one permanent server. Instead, it creates a tiny temporary server for each request. If 1,000 students scan at the same time, 1,000 servers spin up instantly. When they're done, they disappear. You never run out of capacity.
Security by Design	Sensitive operations (password checking, payment processing) happen server-side in API routes. The user's browser never sees database credentials or internal logic.

3. Technology Stack — What Tools We Used and Why

Core Technologies

Technology	What It Is (Simple)	What It Is (Technical)	Why We Chose It
Next.js 16	A framework for building websites	A React-based full-stack framework with file-system routing, server-side rendering, and built-in API routes	Lets us build both the frontend AND backend in one project. No need for a separate Express.js server.
React 19	A library for building interactive UIs	A declarative, component-based	Industry standard. Components make UI

Technology	What It Is (Simple)	What It Is (Technical)	Why We Chose It
		JavaScript library for building user interfaces	reusable and maintainable.
TypeScript	JavaScript but with safety checks	A statically-typed superset of JavaScript	Catches bugs at compile time instead of runtime. If you accidentally pass a number where a string is expected, TypeScript yells at you <i>before</i> the user sees a crash.
Tailwind CSS v4	A utility-first CSS framework	A PostCSS-based utility framework that generates only the CSS classes you actually use	Lets us style directly in our HTML/JSX without writing separate CSS files. Results in tiny CSS bundles.
Prisma ORM	A tool for talking to databases	An Object-Relational Mapping library that provides type-safe database queries	Instead of writing raw SQL like <code>SELECT * FROM users WHERE id = '123'</code> , we write <code>prisma.user.findUnique({ where: { id: '123' } })</code> . It's safer (prevents SQL injection) and gives us autocomplete in our code editor.
PostgreSQL	A database	An open-source relational database management system (RDBMS)	The gold standard for production applications. Supports complex queries, transactions, and concurrent connections.
NextAuth.js v4	A login system	A complete authentication solution for Next.js supporting multiple providers (Google, GitHub, Email, Credentials)	Handles the entire login flow: session management, JWT tokens, cookie security, CSRF protection — all out of the box.
bcrypt	A password scrambler	A cryptographic hashing function based on the Blowfish cipher	Converts plain-text passwords into irreversible hashes. Even if someone steals the database, they cannot recover the original passwords.

Technology	What It Is (Simple)	What It Is (Technical)	Why We Chose It
Nodemailer	An email sender	A Node.js module for sending emails via SMTP	Sends beautifully formatted HTML email receipts through Gmail's servers.
qrcode	A QR code generator	A library that converts text strings into QR code images (Data URLs)	Generates the student's unique boarding pass QR code entirely on the client side.
html5-qrcode	A QR code camera reader	A library that accesses the device camera and decodes QR codes in real-time	Turns any smartphone camera into a QR code scanner. No app download needed.

How These Technologies Relate

```

graph LR
    subgraph "Frontend (Client-Side)"
        React --> TailwindCSS
        React --> QRCode["qrcode (generates QR)"]
        React --> H5QR["html5-qrcode (reads QR)"]
        React --> NextAuth_Client["NextAuth React Hooks"]
    end

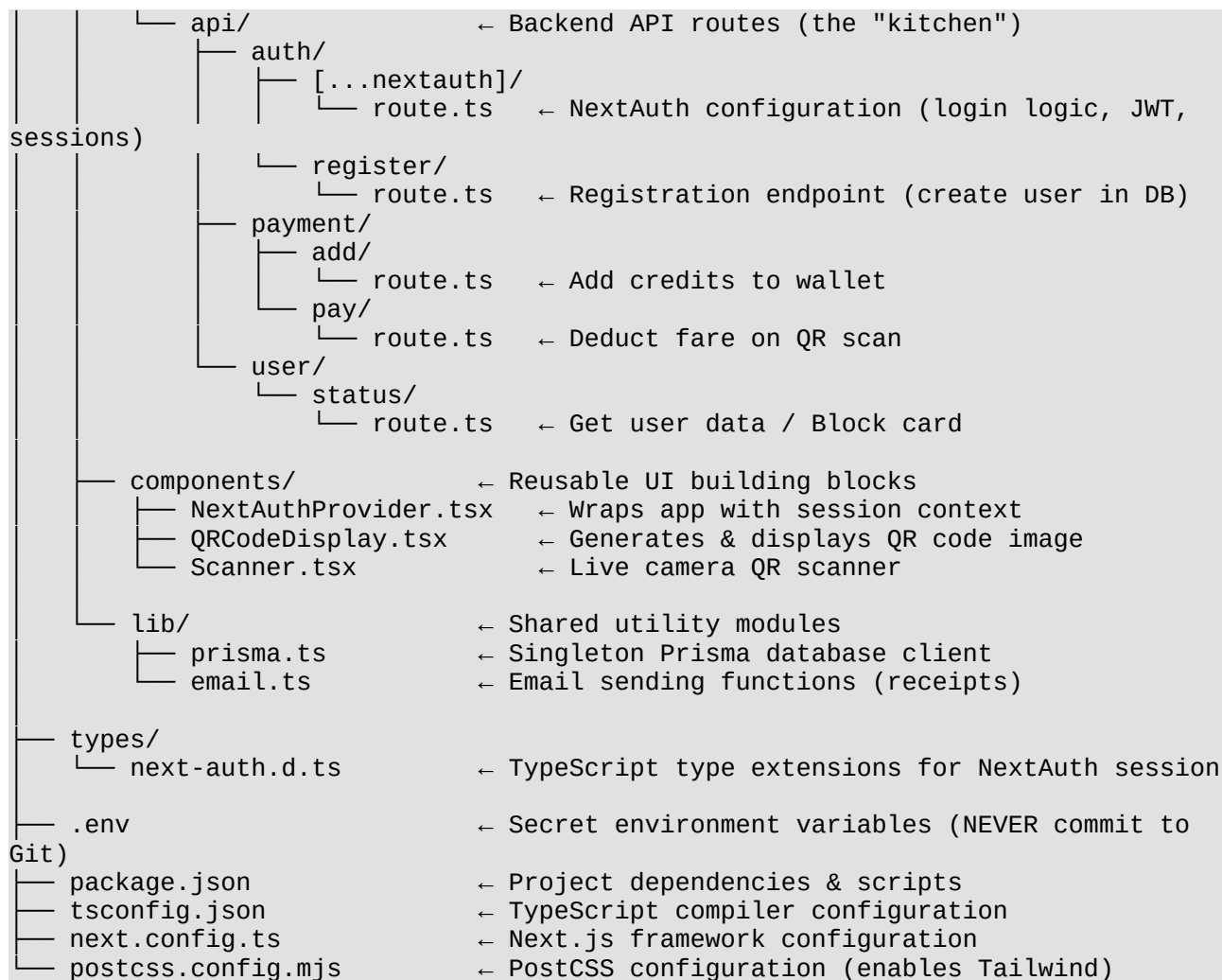
    subgraph "Backend (Server-Side)"
        NextJS_API["Next.js API Routes"] --> Prisma
        NextJS_API --> Bcrypt["bcrypt (password hashing)"]
        NextJS_API --> Nodemailer
        NextJS_API --> NextAuth_Server["NextAuth Server"]
    end

    Prisma --> PostgreSQL
    Nodemailer --> Gmail
    NextAuth_Client -. "HTTP Session Cookie" .-> NextAuth_Server
  
```

4. Project File Structure — Where Everything Lives

```

Bus-Credit-System-BCS--main/
├── prisma/
│   └── schema.prisma      ← Database blueprint (defines User & Transaction
tables)
├── src/
│   ├── app/               ← Next.js App Router (pages & API routes)
│   │   └── layout.tsx     ← Root layout (wraps EVERY page with font, theme,
auth)
│   ├── page.tsx           ← Main dashboard (what you see after login)
│   ├── globals.css        ← Global styles (theme colours, loader animation,
scrollbar)
│   ├── login/
│   │   └── page.tsx       ← Login page (Student/Driver tabs)
│   └── register/
│       └── page.tsx       ← Registration page (create new accounts)
  
```



Why is the structure like this?

Next.js uses a concept called **file-system routing**. This means: **the file path = the URL path**.

File Path	URL the User Visits
src/app/page.tsx	https://busit-web.vercel.app/ (the homepage/dashboard)
src/app/login/page.tsx	https://busit-web.vercel.app/login
src/app/register/page.tsx	https://busit-web.vercel.app/register
src/app/api/auth/register/route.ts	POST https://busit-web.vercel.app/api/auth/register
src/app/api/payment/pay/route.ts	POST https://busit-web.vercel.app/api/payment/pay

You don't need to write any routing configuration. Just create a file in the right folder and Next.js automatically makes it accessible at that URL. This is one of the biggest advantages of Next.js over raw React.

5. The Frontend — What the User Sees

5.1 Root Layout ([layout.tsx](#))

What it does: This is the "wrapper" that surrounds every single page. Think of it like the walls, floor, and ceiling of a building — every room (page) is inside it.

Key responsibilities:

1. **Loads the Inter font** from Google Fonts for consistent, modern typography across the app.
2. **Sets the dark theme** (`className="dark"` on `<html>`) and applies the dark background colour (`bg-zinc-950`).
3. **Wraps everything in `<NextAuthProvider>`** — this is critical. It makes the user's login session available to every page and component in the app. Without this wrapper, no page would know if the user is logged in or not.

Simple analogy: Layout is like the frame of a picture. The picture inside changes (login page, dashboard, etc.), but the frame (dark theme, font, auth context) stays the same.

5.2 Login Page ([login/page.tsx](#))

What the user sees: A clean dark card with "BUSIT Login" at the top, two tabs (Student / Driver), a Student ID input, a Password input, and a "Sign In" button.

What happens when they click "Sign In":

```
sequenceDiagram
    participant User as User's Browser
    participant NextAuth as NextAuth.js (Server)
    participant DB as PostgreSQL Database

    User->>NextAuth: signIn('credentials', { studentId, password })
    NextAuth->>DB: prisma.user.findUnique({ where: { studentId } })
    DB-->>NextAuth: Returns user record (with hashed password)
    NextAuth->>NextAuth: bcrypt.compare(plainPassword, hashedPassword)
    alt Password matches
        NextAuth-->>User: □ Set session cookie, redirect to dashboard
    else Password does not match
        NextAuth-->>User: □ Return error: "Invalid credentials"
    end
end
```

Step by step:

1. The user types their Student ID and password and clicks "Sign In".
2. The `signIn()` function from NextAuth sends the credentials to the server.
3. On the server, the `authorize()` function in `[...nextauth]/route.ts` runs.
4. It queries the database for a user with that Student ID.
5. If found, it uses `bcrypt.compare()` to check if the typed password matches the stored hash.
6. If everything checks out, NextAuth creates a **JWT token** (a cryptographically signed string containing the user's ID, role, and student ID), stores it in a **secure HTTP-only cookie** in the

browser, and redirects the user to the dashboard.

7. If anything fails, the user sees "Invalid ID or Password".

Why `redirect: false`? By default, NextAuth redirects the user itself. We set `redirect: false` so we can handle errors in our own UI (show a red error box) instead of NextAuth redirecting to a generic error page.

5.3 Registration Page ([register/page.tsx](#))

What it does: Lets new users create an account. They choose Student or Driver, fill in their ID, name, email, and password, then click "Register".

What happens behind the scenes:

1. The form data is sent as a POST request to `/api/auth/register`.
2. The server checks if a user with that Student ID or email already exists.
3. The password is hashed with bcrypt (10 salt rounds) before being stored.
4. A new user record is created in the database with 0 credits and an "ACTIVE" card status.
5. On success, the user is redirected to the login page.

5.4 Dashboard ([page.tsx](#))

This is the main screen and it behaves **completely differently** depending on whether you are a Student or a Driver.

Student Dashboard

What the student sees:

- **Header:** "BUSIT" brand name + "Student Wallet" label + Logout button
- **Welcome section:** Their name and Student ID
- **Balance card:** Shows current credits (e.g., "180 CR") and card status (ACTIVE/BLOCKED)
- **QR Code:** Their unique boarding pass — a scannable QR code containing their Student ID
- **Action buttons:** "Add Credits" and "Block Card"
- **Transaction history:** A scrollable list of their 10 most recent top-ups and rides

Driver Dashboard

What the driver sees:

- **Header:** "BUSIT" brand name + "Driver Portal" label + Logout button
- **Welcome section:** Their name and Driver ID
- **Live Camera Scanner:** A real-time camera feed that continuously scans for QR codes

How does the dashboard know which view to show?


```
const isDriver = session.user.role === 'DRIVER';
```

This single line checks the user's role from their session. If they're a driver, it renders the `<Scanner />` component. If they're a student, it renders the balance card, QR code, and transaction history. This is called **conditional rendering** — the same page shows different content based on who is logged in.

How does the dashboard load data?

When the dashboard first opens, it runs `fetchUserData()` which calls `GET /api/user/status`. This API route:

1. Checks the user's session to get their Student ID.
2. Queries the database for their full profile including the last 10 transactions.
3. Returns all this data as JSON, which React then renders on screen.

5.5 The 'use client' Directive

You'll notice every frontend file starts with `'use client';`. In Next.js, components are **server components** by default (they run on the server and send static HTML to the browser). But our pages need interactivity — form inputs, button clicks, camera access, state management. The `'use client'` directive tells Next.js: "This component needs to run in the browser, not on the server."

6. The Backend — What Happens Behind the Scenes

All backend logic lives in the `src/app/api/` directory. Each `route.ts` file is a standalone HTTP endpoint (a URL the frontend can call).

6.1 Registration API ([api/auth/register/route.ts](#))

URL: `POST /api/auth/register` **Who can access it:** Anyone (no authentication required — you can't require a login to create a login)

What it does, line by line:

1. Receives the request body: `{ studentId, name, email, password, role }`
2. **Validation:** Checks that all required fields are present. If not → `400 Bad Request`.
3. **Duplicate check:** Queries the database for any existing user with the same Student ID OR email. If found → `400 User already exists`.
4. **Password hashing:** `bcrypt.hash(password, 10)` — this converts the plain-text password into a hash. The `10` is the "salt rounds" (how many times the algorithm scrambles the password). Higher = slower but more secure. `10` is the industry standard.
5. **User creation:** Inserts a new row into the `User` table with the hashed password, role, `0` credits, and `"ACTIVE"` card status.
6. Returns `{ success: true }`.

6.2 Authentication API ([api/auth/\[...nextauth\]/route.ts](#))

URL: POST /api/auth/callback/credentials (and many other NextAuth routes) **What is [...nextauth]?** This is a Next.js "catch-all" route. The [...] syntax means it matches *any* URL that starts with /api/auth/. NextAuth needs many sub-routes (signin, signout, callback, session, csrf) and this one file handles all of them.

The authOptions configuration explained:

```
export const authOptions: NextAuthOptions = {
  providers: [
    CredentialsProvider({
      // This tells NextAuth: "Users will log in with a username and password,
      // not with Google or GitHub OAuth."
      credentials: {
        studentId: { label: "Student ID", type: "text" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        // This function runs every time someone tries to log in.
        // It's our "bouncer" – it decides if the credentials are valid.

        // Step 1: Find the user in the database by Student ID
        const user = await prisma.user.findUnique({
          where: { studentId: credentials.studentId }
        });

        // Step 2: Compare the provided password with the stored hash
        const isValid = await bcrypt.compare(
          credentials.password, // what the user typed
          user.password         // the hash stored in the database
        );

        // Step 3: If valid, return the user object. NextAuth will create a
        session.
        // If invalid, throw an error. NextAuth will reject the login.
        return { id: user.id, name: user.name, role: user.role, studentId:
user.studentId };
      }
    )
  ],
  session: {
    strategy: "jwt"
    // "jwt" means we store session data in a cryptographically signed token
    // inside a cookie, NOT in a server-side database.
    // This is ideal for serverless because there's no persistent server memory.
  },
  callbacks: {
    // These callbacks let us customize what data goes into the JWT and session.
    async jwt({ token, user }) {
      // When the user first logs in, `user` is available.
      // We add our custom fields (role, studentId) to the JWT token.
      if (user) {
        token.role = user.role;
        token.studentId = user.studentId;
      }
      return token;
    },
    async session({ session, token }) {
      // Every time the frontend calls useSession() or getSession(),
      // this callback runs. We copy our custom fields from the JWT
      // into the session object so the frontend can access them.
    }
  }
}
```

```

    session.user.role = token.role;
    session.user.studentId = token.studentId;
    return session;
  }
}
};

```

6.3 Payment API — Fare Deduction ([api/payment/pay/route.ts](#))

URL: POST /api/payment/pay **Who can access it:** Only authenticated Drivers

This is the most critical endpoint in the entire system. It processes the actual fare payment.

Step-by-step flow:

1. **Authorization check:** Verifies the caller is logged in AND has the DRIVER role. A student cannot call this endpoint.
2. **Receives the Student ID** from the scanned QR code.
3. **Looks up the student** in the database.
4. **Validates:** Is the student's card ACTIVE? Do they have at least 20 credits?
5. **Processes the payment in a database transaction:**

```

await prisma.$transaction([
  prisma.user.update({
    where: { id: student.id },
    data: { credits: { decrement: FARE } } // Subtract 20 credits
  }),
  prisma.transaction.create({
    data: {
      userId: student.id,
      amount: -FARE, // Negative because it's a deduction
      type: 'PAYMENT'
    }
  })
]);

```

What is \$transaction and why is it crucial?

A database transaction ensures that BOTH operations (deducting credits AND creating the transaction record) either BOTH succeed or BOTH fail. Imagine if the credits were deducted but the transaction record failed to save — the student would lose money with no proof of the ride. A transaction prevents this "partial failure" scenario. It's all-or-nothing.

1. **Sends an email receipt** asynchronously (fire-and-forget so the driver isn't kept waiting).
2. **Returns success** to the driver's scanner.

6.4 Payment API — Add Credits ([api/payment/add/route.ts](#))

URL: POST /api/payment/add **Who can access it:** Only authenticated Students

Similar to the pay route, but in reverse — it *adds* credits instead of deducting them. It also uses a `$transaction` to atomically increment the balance and create a `TOP_UP` transaction record, then sends an email receipt.

6.5 User Status API ([api/user/status/route.ts](#))

URL: GET /api/user/status — Fetch user profile and transaction history **URL:** POST /api/user/status — Block or unblock a card

The GET handler is called every time the dashboard loads. It returns the user's name, student ID, credit balance, card status, and the 10 most recent transactions (sorted newest first).

The POST handler lets a student block their own card (like freezing a bank card if it's lost). A blocked card cannot be scanned — the pay endpoint checks `cardStatus !== 'ACTIVE'` and rejects the payment.

7. The Database — Where Data is Stored

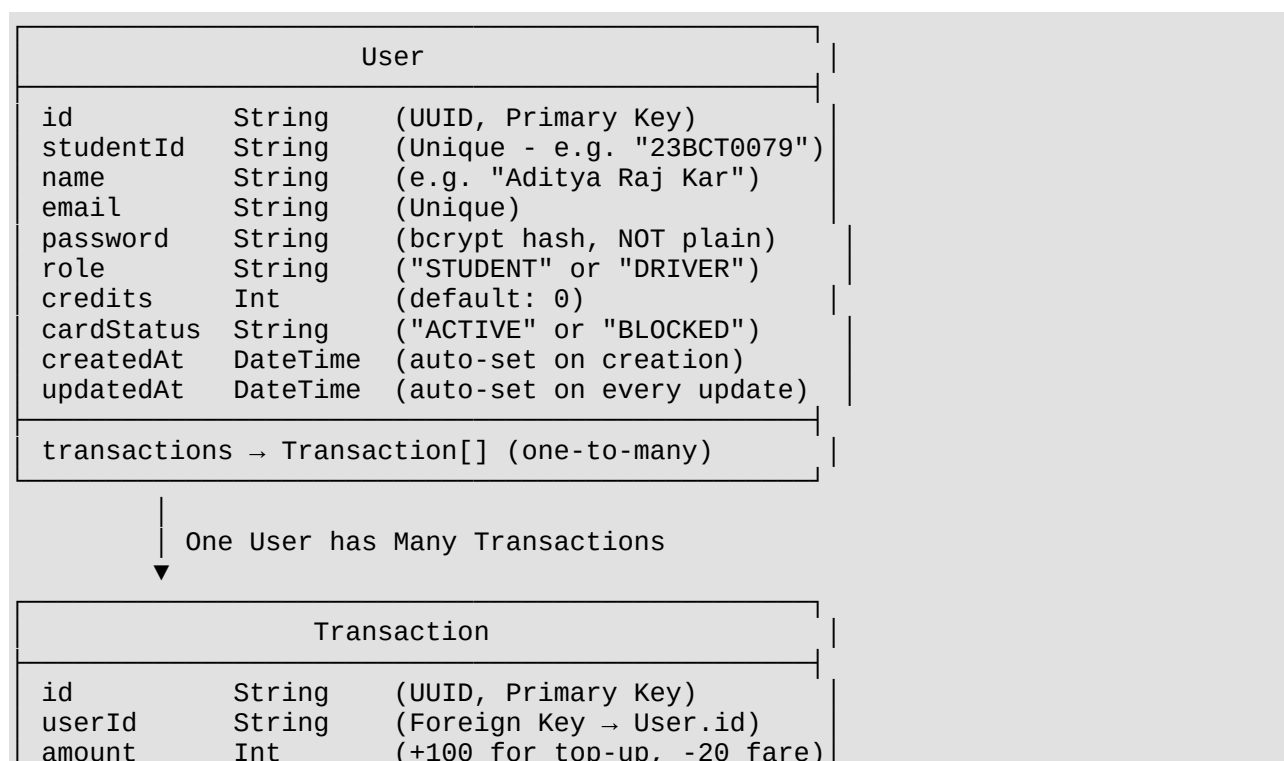
7.1 Database Provider: Neon PostgreSQL

The database runs on [Neon](#), a serverless PostgreSQL provider. "Serverless" means:

- You don't manage a database server yourself.
- The database automatically scales up when traffic increases and scales down (to save costs) when idle.
- It supports "connection pooling" — efficiently managing thousands of simultaneous connections from Vercel's serverless functions.

7.2 The Schema ([prisma/schema.prisma](#))

The Prisma schema is the **single source of truth** for the database structure. It defines what tables exist, what columns they have, and how they relate to each other.



type	String	("PAYMENT" or "TOP_UP")
timestamp	DateTime	(auto-set on creation)

7.3 Key Database Concepts

Primary Key (@id): A unique identifier for each row. We use UUIDs (e.g., "a3f8c2b1-7d4e-4a9b-8f6c-1e2d3a4b5c6d") instead of auto-incrementing integers. UUIDs are better for distributed systems because they're globally unique — you never get ID collisions even if two servers create records at the exact same millisecond.

Unique Constraints (@unique): Both `studentId` and `email` are marked unique. The database will physically refuse to create two users with the same Student ID or email. This is a *database-level* guarantee — even if our application code has a bug, the database itself prevents duplicates.

Foreign Key (@relation): The `userId` field in `Transaction` points to a `User`. This creates a formal relationship: every transaction *belongs to* exactly one user, and every user *has many* transactions. Prisma uses this to let us write `include: { transactions: true }` to automatically fetch a user's transactions alongside their profile.

Default Values (@default): New users automatically get `credits: 0`, `cardStatus: "ACTIVE"`, and `createdAt: now()`. This means we don't have to manually set these when creating a user — the database handles it.

7.4 The Prisma Client Singleton ([lib/prisma.ts](#))

```
const globalForPrisma = global as unknown as { prisma: PrismaClient }

export const prisma =
  globalForPrisma.prisma ||
  new PrismaClient({ log: ['query'] })

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
```

Why this pattern? During development, Next.js's "hot reloading" feature restarts the server code every time you save a file. Each restart would create a NEW database connection. After 50 saves, you'd have 50 connections, and the database would refuse any more. This "singleton" pattern stores the Prisma client in Node.js's `global` object so it survives hot reloads. In production, this isn't needed (Vercel creates fresh instances), but it doesn't hurt.

8. Authentication & Security — Keeping Users Safe

8.1 Password Hashing with bcrypt

The problem: If we stored passwords as plain text (e.g., "mypassword123"), a database breach would expose every user's password. Since people reuse passwords across sites, this would compromise their bank accounts, emails, and social media too.

The solution: We use `bcrypt`, a one-way hashing algorithm.

Input: "mypassword123"

```
Output: "$2b$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhwy"
```

Key properties of bcrypt:

- **One-way:** You cannot reverse the hash back to the original password. There is no "decrypt" function.
- **Salted:** Each hash includes random data (a "salt"), so two users with the same password get *different* hashes. This prevents attackers from using precomputed "rainbow tables".
- **Slow by design:** The 10 salt rounds make the hashing intentionally slow (~100ms). This is great for security because an attacker trying to brute-force millions of passwords would take years.

How login verification works:

```
const isPasswordValid = await bcrypt.compare("mypassword123", storedHash);  
// bcrypt internally extracts the salt from the stored hash,  
// re-hashes the input with that same salt, and checks if they match.
```

8.2 JWT Sessions

What is a JWT? A JSON Web Token is a string that looks like this:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEyMyIsInJvbmGUiOiJTVFVERU5UIIn0.abc123signature
```

It has three parts separated by dots:

1. **Header:** Algorithm used to sign it (HS256)
2. **Payload:** The actual data (user ID, role, student ID)
3. **Signature:** A cryptographic signature created using the NEXTAUTH_SECRET

Why JWTs? In a serverless environment (Vercel), there is no persistent server memory. Each API request might be handled by a different server. We can't store sessions in memory. JWTs solve this because the session data is stored *in the cookie itself* — every request carries its own proof of identity.

Security of JWTs: The signature prevents tampering. If someone modifies the payload (e.g., changes their role from "STUDENT" to "DRIVER"), the signature won't match, and the server will reject it. Only someone who knows the NEXTAUTH_SECRET can create a valid signature.

8.3 Route Protection

Every protected API route follows this pattern:

```
const session = await getSession(authOptions);  
if (!session) {  
  return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });  
}
```

Additionally, role-based checks are applied:

- **Pay endpoint:** Only drivers can process payments (`session.user.role !== 'DRIVER'`)

- **Add credits endpoint:** Only students can add credits (`session.user.role !== 'STUDENT'`)

8.4 TypeScript Type Augmentation ([types/next-auth.d.ts](#))

By default, NextAuth's `Session` type only knows about `name`, `email`, and `image`. Our app needs `id`, `role`, and `studentId`. This file extends NextAuth's TypeScript types to include our custom fields, giving us autocomplete and type safety when we access `session.user.role`.

9. The QR Code System — The Core Innovation

The QR code system is what makes BUSIT unique. It replaces physical hardware (barcode readers, NFC cards) with a pure software solution.

9.1 QR Code Generation ([QRCodeDisplay.tsx](#))

How it works:

```
QRCode.toDataURL(studentId, {  
  width: 200,  
  margin: 2,  
  color: { dark: '#0f172a', light: '#ffffff' }  
})
```

1. The `qrcode` library takes the student's ID string (e.g., "23BCT0079") and encodes it into a QR code image.
2. The image is generated as a **Data URL** — a base64-encoded image string that can be used directly as an `<img>` source without needing a separate image file.
3. The QR code is generated entirely **client-side** in the browser. It doesn't make any network request.
4. The QR code is displayed inside a white container (for contrast) on the student's dashboard.

What data is in the QR code? Just the student's unique ID string (e.g., "23BCT0079"). Nothing sensitive like passwords or email addresses. The ID is used by the backend to look up the student in the database.

9.2 QR Code Scanning ([Scanner.tsx](#))

How it works:

```
sequenceDiagram  
    participant Camera as Driver's Camera  
    participant Scanner as Html5QrcodeScanner  
    participant API as /api/payment/pay  
    participant DB as Database  
    participant Email as Gmail SMTP  
  
    Camera->>Scanner: Continuous video feed  
    Scanner->>Scanner: Decode QR code frame-by-frame (10 FPS)  
    Scanner->>Scanner: QR detected! Extract "23BCT0079"  
    Scanner->>API: POST { studentId: "23BCT0079" }
```

```
API->>DB: Find student, check balance, deduct 20 CR
DB-->>API: □ Updated
API->>Email: Send receipt (async, fire-and-forget)
API-->>Scanner: { success: true, message: "Payment Successful!" }
Scanner-->>Camera: Show green "Payment Successful!" message
Note over Scanner: Wait 3 seconds, then reset for next scan
```

Technical details:

1. `Html5QrcodeScanner` is initialized with `fps: 10` — it analyses 10 camera frames per second looking for QR codes.
 2. `qrbox: { width: 250, height: 250 }` draws a targeting box in the centre of the camera view. The scanner only looks for QR codes within this box (improves accuracy and performance).
 3. When a QR code is detected, the `onSuccess` callback fires with the decoded text.
 4. The scanner immediately pauses itself (`scanner.pause(true)`) to prevent double-scanning.
 5. The decoded Student ID is sent to `POST /api/payment/pay`.
 6. The result (success or error) is displayed on screen for 3 seconds.
 7. A "Scan Next" button appears so the driver can reload and scan the next student.
-

10. The Payment Flow — How Money Moves

10.1 Adding Credits (Top-Up)

```
sequenceDiagram
    participant Student as Student's Browser
    participant API as /api/payment/add
    participant DB as Database
    participant Email as Gmail

    Student->>Student: Clicks "Add Credits", enters 100
    Student->>API: POST { amount: 100 }
    API->>API: Verify session (is this a STUDENT?)
    API->>DB: Find user, verify they exist
    API->>DB: BEGIN TRANSACTION
    API->>DB: UPDATE User SET credits = credits + 100
    API->>DB: INSERT Transaction (amount: 100, type: TOP_UP)
    API->>DB: COMMIT TRANSACTION
    API->>Email: sendTopUpReceipt() [async]
    API-->>Student: { success: true }
    Student->>Student: Refresh dashboard (shows new balance)
```

10.2 Paying Fare (Deduction)

```
sequenceDiagram
    participant Driver as Driver's Scanner
    participant API as /api/payment/pay
    participant DB as Database
    participant Email as Gmail
```



```

Driver->>Driver: Camera detects QR code "23BCT0079"
Driver->>API: POST { studentId: "23BCT0079" }
API->>API: Verify session (is this a DRIVER?)
API->>DB: Find student by studentId
API->>API: Check: cardStatus === 'ACTIVE'?
API->>API: Check: credits >= 20?
API->>DB: BEGIN TRANSACTION
API->>DB: UPDATE User SET credits = credits - 20
API->>DB: INSERT Transaction (amount: -20, type: PAYMENT)
API->>DB: COMMIT TRANSACTION
API->>Email: sendPaymentReceipt() [async]
API-->>Driver: { success: true, message: "Payment Successful!" }

```

10.3 Why Transactions Are Stored with Positive AND Negative Amounts

Event	amount stored	type
Student adds 100 credits	+100	TOP_UP
Student rides shuttle	- 20	PAYMENT

This design makes it easy to:

- Calculate total spent: `SUM(amount) WHERE type = 'PAYMENT'`
- Show transaction history with clear +/- indicators
- Audit the complete financial trail

11. The Email Notification System

11.1 How it works ([lib/email.ts](#))

The email system uses **Nodemailer** to send emails through **Gmail's SMTP** (Simple Mail Transfer Protocol) servers.

SMTP in simple terms: When you press "Send" on an email, your email client doesn't deliver it directly to the recipient. It hands it off to an SMTP server (like a post office), which routes and delivers it. Our app uses Gmail's "post office" to send emails on behalf of the BUSIT system.

The transporter configuration:

```

const transporter = nodemailer.createTransport({
  service: 'gmail',          // Use Gmail's SMTP servers
  auth: {
    user: process.env.SMTP_EMAIL,    // The Gmail address sending the emails
    pass: process.env.SMTP_PASSWORD, // A 16-character "App Password" (NOT the
    // Gmail password)
  },
});

```

Why an App Password? Google doesn't allow applications to use your regular Gmail password (for security). Instead, you generate a special 16-character password from your Google Account settings that is used exclusively by this app. If the app is compromised, you can revoke just this app password without changing your main password.

11.2 Two Types of Emails

Email Type	Trigger	Key Info Displayed
Ride Receipt (sendPaymentReceipt)	Driver scans a student's QR code	Date, fare deducted, remaining balance, low balance warning
Top-Up Receipt (sendTopUpReceipt)	Student adds credits to their wallet	Date, amount added, new balance

Both emails are **HTML-formatted** with inline CSS styling (because email clients don't support external stylesheets). They feature the BUSIT branding, a dark header, and a clean receipt layout.

11.3 Asynchronous "Fire and Forget"

```
sendPaymentReceipt(student.email, student.name, FARE, student.credits - FARE)
  .catch(console.error);
```

Notice we do NOT use `await`. The email is sent asynchronously — the API response is returned to the driver immediately, and the email is sent in the background. This means:

- The driver doesn't wait 2-3 seconds for the email to send before seeing "Payment Successful!"
- If the email fails (e.g., Gmail is temporarily down), the payment still succeeds. The `.catch(console.error)` logs the error but doesn't crash the endpoint.

12. Deployment & Infrastructure

12.1 The Deployment Stack

```
graph LR
  A["Your Computer<br/>(Development)"] -->|"git push"| B["GitHub<br/>(Code Repository)"]
  B -->|"Auto-deploy on push"| C["Vercel<br/>(Hosting Platform)"]
  C -->|"DATABASE_URL"| D["Neon<br/>(PostgreSQL Database)"]
  C -->|"SMTP credentials"| E["Gmail<br/>(Email Service)"]
```

12.2 How Vercel Deployment Works

1. You push code to GitHub.
2. Vercel detects the push (via a webhook GitHub sends to Vercel).
3. Vercel clones your repository into a temporary build environment.
4. It runs `npm install` (which triggers `postinstall: prisma generate` — generating the Prisma client).
5. It runs `npm run build` (next build) which:
 - Compiles TypeScript to JavaScript
 - Pre-renders static pages (login, register)

- Bundles API routes into serverless functions
6. The build output is deployed to Vercel's global CDN (Content Delivery Network) across data centres worldwide.
 7. Each API route becomes an independent serverless function that spins up on demand.

12.3 Environment Variables

Environment variables are secrets that the code needs but should NEVER be committed to Git. They are configured in Vercel's dashboard.

Variable	Purpose	Example
DATABASE_URL	Connection string for the Neon PostgreSQL database	postgresql://user:pass@host/db?sslmode=require
NEXTAUTH_SECRET	Secret key used to sign JWT tokens	supersecret123!@#
SMTP_EMAIL	Gmail address used as the "from" address for emails	busit.system@gmail.com
SMTP_PASSWORD	Gmail App Password for SMTP authentication	abcdefghijklmnop

12.4 The `postinstall` Script

```
"postinstall": "prisma generate"
```

This is a critical detail. When Vercel runs `npm install`, it needs to generate the Prisma Client (the TypeScript code that knows how to talk to your specific database schema). Without this script, the build would fail with "PrismaClient is not generated" because Prisma needs to read your `schema.prisma` and generate type-safe query methods before the Next.js build can compile the API routes.

13. Complete Request Lifecycle — A Full Journey

Let's trace what happens when a Driver scans a Student's QR code, from the photon hitting the camera sensor to the email landing in the inbox.

Step 1: Camera Captures QR Code

The `html5-qrcode` library captures video frames from the device camera at 10 FPS and analyses each frame for QR code patterns.

Step 2: QR Code Decoded

The library decodes the QR code and extracts the string `"23BCT0079"`. The `onSuccess` callback in `Scanner.tsx` fires.

Step 3: Frontend Sends API Request

```
fetch('/api/payment/pay', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ studentId: "23BCT0079" })
})
```

The browser automatically includes the NextAuth session cookie with this request.

Step 4: DNS Resolution

The browser resolves `busit-web.vercel.app` to Vercel's CDN IP address.

Step 5: TLS Handshake

The browser and Vercel establish an encrypted HTTPS connection. All data is encrypted in transit.

Step 6: Vercel Routes the Request

Vercel's edge network receives the request, identifies it as an API route (`/api/payment/pay`), and spins up a serverless function (a tiny Node.js process) to handle it.

Step 7: NextAuth Session Verification

`getSession(authOptions)` reads the JWT cookie, verifies its signature using `NEXTAUTH_SECRET`, decodes the payload, and returns the session object with `{ user: { id, role: 'DRIVER', studentId } }`.

Step 8: Authorization Check

The code checks `session.user.role !== 'DRIVER'`. Since the user IS a driver, it passes.

Step 9: Database Query — Find Student

Prisma sends a SQL query to Neon PostgreSQL:

```
SELECT * FROM "User" WHERE "studentId" = '23BCT0079'
```

The connection travels over the internet from Vercel's data centre to Neon's PostgreSQL server (encrypted with SSL).

Step 10: Business Logic Validation

The server checks: Is the student's card ACTIVE? Do they have ≥ 20 credits? Both pass.

Step 11: Database Transaction — Process Payment

Prisma sends two SQL queries wrapped in a transaction:

```
BEGIN;
UPDATE "User" SET credits = credits - 20, "updatedAt" = NOW() WHERE id = '...';
INSERT INTO "Transaction" (id, "userId", amount, type, timestamp) VALUES ('...',
'...', -20, 'PAYMENT', NOW());
COMMIT;
```

Step 12: Email Dispatch (Async)

The `sendPaymentReceipt` function is called **WITHOUT** `await`. It creates an SMTP connection to Gmail's servers (`smtp.gmail.com:465`), authenticates with the App Password, and sends the HTML email. This happens in the background.

Step 13: API Response

The serverless function returns:

```
{ "success": true, "message": "Payment Successful!" }
```

Step 14: Frontend Updates

The Scanner component receives the response, displays a green "Payment Successful!" banner, and after 3 seconds, shows a "Scan Next" button.

Step 15: Student Receives Email

Gmail's servers deliver the HTML receipt to the student's inbox within seconds.

Total time from scan to response: ~200-500ms (excluding email delivery which happens asynchronously).

14. Interview Questions & Answers

Architecture & Design

Q: Why did you choose Next.js over a separate React frontend + Express backend?

A: Next.js provides a "full-stack" solution where both the frontend and backend live in the same project. This has several advantages:

- **Single deployment:** One `git push` deploys both UI and API. No need to coordinate two separate deployments.
- **Shared types:** TypeScript types defined in one place are used by both frontend and backend, reducing bugs.
- **File-system routing:** URLs are determined by file paths, eliminating manual routing configuration.
- **Serverless-ready:** Next.js API routes are designed to run as serverless functions on Vercel, making scaling effortless.

The tradeoff is that a monolithic Next.js app is harder to scale independently (you can't scale just the backend without the frontend). But for a project of this size, the simplicity far outweighs this concern.

Q: Why PostgreSQL over MongoDB or MySQL?

A: PostgreSQL was chosen because:

- **ACID compliance:** Our payment system requires transactions (debiting one account and recording it atomically). PostgreSQL's ACID guarantees ensure data consistency even under concurrent access. MongoDB's document model makes multi-document transactions more complex.
 - **Relational data:** Users and Transactions have a clear one-to-many relationship. Relational databases excel at this.
 - **Prisma compatibility:** Prisma has first-class PostgreSQL support with features like `@default(uuid())` and `$transaction`.
 - **Neon integration:** Neon provides serverless PostgreSQL specifically optimized for platforms like Vercel, with built-in connection pooling.
-

Q: What is the role of the Prisma ORM? Could you use raw SQL instead?

A: Prisma serves three purposes:

1. **Type safety:** It generates TypeScript types from the schema, so `prisma.user.findUnique()` returns a properly typed `User` object. With raw SQL, you'd get untyped any results.
2. **SQL injection prevention:** Prisma automatically parameterizes queries. With raw SQL, a malicious student ID like `' ; DROP TABLE User ; --` could destroy the database. Prisma makes this impossible.
3. **Migration management:** Prisma can generate and apply database migrations, making schema changes safe and trackable.

Yes, raw SQL would work, but it would require manual type definitions, manual parameterization, and manual migration management.

Security

Q: How are passwords protected in this system?

A: Passwords are never stored in plain text. When a user registers, their password is hashed using the bcrypt algorithm with 10 salt rounds. Bcrypt is specifically designed for password hashing because:

- It's intentionally slow (~100ms per hash), making brute-force attacks impractical.
- Each hash includes a random salt, so identical passwords produce different hashes.
- It's a one-way function — there is no way to reverse a hash back to the original password.

During login, `bcrypt.compare()` re-hashes the input with the stored salt and checks if the result matches. The plain-text password is never stored or logged.

Q: What is a JWT and why did you use it instead of server-side sessions?

A: A JWT (JSON Web Token) is a cryptographically signed token that contains encoded user data (ID, role, student ID). We chose JWTs because:

- **Serverless compatibility:** Vercel's serverless functions are stateless — they don't have shared memory. Server-side sessions require a session store (like Redis), adding complexity. JWTs are self-contained; the session data travels with each request in a cookie.
- **No database lookup per request:** With server-side sessions, every API call requires a database query to validate the session. With JWTs, the server just verifies the signature mathematically — no database round-trip needed.
- **Scalability:** Since JWTs are stateless, any server can verify them. There's no "session affinity" requirement.

The tradeoff is that JWTs cannot be individually revoked (you can't "log out" a specific JWT without waiting for it to expire or maintaining a blocklist). For this application, the short session duration mitigates this risk.

Q: How do you prevent a student from calling the payment endpoint to give themselves free credits?

A: Role-based access control (RBAC). Every protected endpoint checks the user's role from their JWT session:

```
if (!session || session.user.role !== 'DRIVER') {  
  return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });  
}
```

The role is set during registration and stored in the database. It's embedded in the JWT during login. Since the JWT is signed with `NEXTAUTH_SECRET`, a user cannot modify their role in the token without invalidating the signature. The server would reject the tampered token.

Q: What prevents double-scanning (charging a student twice for one ride)?

A: The Scanner component implements a state-based guard:

```
if (!loading && !scannedId) {  
  setScannedId(decodedText);  
  handlePayment(decodedText);  
  scanner.pause(true);  
}
```

Once a QR code is decoded, `scannedId` is set and the scanner is paused. The `if (!loading && !scannedId)` condition prevents any subsequent scan callbacks from triggering until the state is reset after a 3-second cooldown.

Database

Q: What is a database transaction and why is it important for payments?

A: A database transaction groups multiple operations into a single atomic unit. Either ALL operations succeed, or ALL are rolled back (undone). In our payment system:

```
await prisma.$transaction([
  prisma.user.update({ ... }),      // Operation 1: Deduct credits
  prisma.transaction.create({ ... }) // Operation 2: Record the payment
]);
```

Without a transaction, if Operation 1 succeeds but Operation 2 fails (e.g., due to a network glitch), the student would lose 20 credits with no record of why. With a transaction, if Operation 2 fails, Operation 1 is automatically rolled back — the credits are restored. This maintains data integrity.

Q: Explain the `@default(uuid())` annotation in the Prisma schema.

A: `@default(uuid())` tells the database to automatically generate a UUID (Universally Unique Identifier) for the `id` field whenever a new row is inserted. UUIDs look like "a3f8c2b1-7d4e-4a9b-8f6c-1e2d3a4b5c6d". We use UUIDs instead of auto-incrementing integers (1, 2, 3, ...) because:

- UUIDs are globally unique across all tables and databases.
 - They don't reveal information (auto-increment IDs leak how many users you have: if your ID is 500, you know there are at least 500 users).
 - They're safe for distributed systems (no central counter needed).
-

Frontend

Q: What is the difference between a Server Component and a Client Component in Next.js?

A: In the Next.js App Router:

- **Server Components** (default) run on the server. They can directly access the database, read files, and fetch data. They send only the final HTML to the browser — no JavaScript is shipped. Great for static content.
- **Client Components** (marked with `'use client'`) run in the browser. They can use React hooks (`useState`, `useEffect`), handle user events (clicks, form submissions), and access browser APIs (camera, `localStorage`).

In BUSIT, all pages are Client Components because they need interactivity: form inputs, button clicks, camera access, session hooks, and dynamic state updates.

Q: Why does `NextAuthProvider` wrap the entire app in `layout.tsx`?

A: `NextAuthProvider` (which is a wrapper around NextAuth's `SessionProvider`) creates a React Context that makes the user's session data available to every component in the app. Without it, calling `useSession()` in any component would return `undefined` because there would be no session context to read from.

By placing it in `layout.tsx` (the root layout), every page and component in the entire app automatically has access to authentication state. This is called the "Provider Pattern" in React.

QR Codes

Q: Why does the QR code only contain the Student ID and not more data?

A: This is a deliberate security and simplicity decision:

- **Security:** If the QR code contained sensitive data (credits, email), anyone who photographs the QR code would gain access to that information.
 - **Freshness:** The Student ID is a permanent identifier. The credit balance changes constantly. If we encoded the balance in the QR code, it would be stale immediately after a transaction.
 - **Server-side validation:** The Student ID is just a lookup key. All actual validation (balance check, card status) happens on the server using real-time database data. This ensures the system is always using the latest information.
-

Q: Can someone screenshot a QR code and use it fraudulently?

A: Currently, yes — this is a known limitation of the prototype. In a production system, you would mitigate this with:

1. **Time-based QR codes:** Generate a new QR code every 30 seconds (like Google Authenticator), encoding the student ID + a timestamp + a cryptographic signature.
 2. **One-time use tokens:** Each QR code contains a single-use token that is invalidated after scanning.
 3. **Geo-fencing:** Only allow scans when the student's device is near a registered bus GPS location.
-

Deployment

Q: What is a serverless function and how does it differ from a traditional server?

A: A traditional server is a computer running 24/7, waiting for requests. You pay for it even when nobody is using your app. A serverless function is a tiny program that exists only when a request arrives. It:

1. **Cold starts:** Spins up in ~50-200ms when a request arrives.
2. **Processes:** Handles the request (e.g., queries the database, sends an email).
3. **Shuts down:** Disappears immediately after responding.

You pay only for the actual execution time (e.g., $200\text{ms} \times \text{number of requests}$). For a university shuttle app that's busy during rush hours and idle at night, serverless is far more cost-effective than a 24/7 server.

Q: What happens if two students are scanned at the exact same millisecond?

A: Vercel spins up two separate serverless function instances — one for each request. They execute in complete isolation. Each function independently:

1. Reads the student's current balance from the database.
2. Deducts 20 credits within a database transaction.

PostgreSQL's transaction isolation ensures that even if both queries hit the database simultaneously, they are serialized — one completes before the other starts. There is no risk of a "race condition" where credits are deducted incorrectly.

Q: How would you add real payments (Stripe) to this system?

A: The integration would follow this flow:

1. Student clicks "Buy 100 Credits" → Frontend creates a Stripe Checkout Session via the backend.
2. Student is redirected to Stripe's hosted payment page (card details never touch our servers).
3. After successful payment, Stripe sends a **Webhook** (a POST request) to our `/api/webhooks/stripe` endpoint.
4. Our webhook handler verifies the event's cryptographic signature (to prevent forged requests), then adds the credits to the student's account.
5. The current "Add Credits" button would be replaced with this flow.

This is the exact next step planned for BUSIT.

Q: What would you improve if you had more time?

A: Several areas:

1. **Real payment integration** (Stripe) to replace the mock credit system.
 2. **Admin dashboard** for university administrators to view all transactions, manage users, and set fare rates.
 3. **Route management** — different fares for different bus routes.
 4. **Rate limiting** on API endpoints to prevent abuse (e.g., someone spamming the register endpoint).
 5. **Time-based QR codes** to prevent screenshot fraud.
 6. **Push notifications** via service workers as an alternative to email.
 7. **Automated testing** — unit tests for API routes, integration tests for the payment flow.
 8. **Proper error monitoring** with a service like Sentry to catch and alert on production errors.
-

End of Documentation

This document covers every file, every function, every data flow, and every design decision in the BUSIT project. It is intended to serve as both a learning resource and an interview preparation guide.